RabbitTClust2: Fast, Scalable, and Versatile Clustering for Massive Genomic Datasets

Tong Zhang [1, †] Xiaoming Xu [2, †] Zekun Yin [1,*] Lifeng Yan[1] Yang Yang[1]
Yijie Gao[1] Xiaohui Duan[1] Bertil Schmidt[3] Weiguo Liu [1,*]
1. School of Software, Shandong University, Jinan, China
{tong.z, lifeng.yan, yang.yang, yijie.gao}@mail.sdu.edu.cn
{zekun.yin, sunrise.duan, weiguo.liu}@sdu.edu.cn
2. Department of Computer Science and Technology, Tsinghua University, Beijing, China
xiaomingxu@tsinghua.edu.cn
3. Institute for Computer Science, Johannes Gutenberg University, Mainz, Germany
bertil.schmidt@uni-mainz.de

***Abstract*—Clustering is a fundamental method for extracting meaningful information from large-scale genomic datasets. As sequencing technologies advance, efficient and scalable clustering tools have become increasingly important. Despite its outstanding efficiency in large-scale genome clustering tasks, RabbitTClust still faces certain limitations. On the one hand, as data volumes continue to grow, there remains room for further optimization of its computational performance. On the other hand, RabbitTClust is not well suited for frequent incremental data updates or fast clustering across multiple thresholds. To address these limitations, we introduce RabbitTClust2, a highly efficient and versatile tool designed for clustering large-scale genomic sequences. RabbitTClust2 integrates an efficient sketching algorithm, a pruning- and inverted-index-based minimum spanning tree construction method, and strategies for reusing intermediate results. With these advancements, RabbitTClust2 is able to cluster the latest RefSeq bacterial dataset (195k genomes, 820 GB in FASTA) within 5 minutes. Compared to previous versions, RabbitTClust2 achieves a 2.4× to 4.5× speedup while maintaining comparable clustering accuracy, with a 21% reduction in memory consumption. On a distributed multi-node platform, RabbitTClust2 is capable of clustering 2.6 million genomes in approximately one hour. Furthermore, RabbitTClust2 offers significant versatility by supporting efficient incremental clustering and rapid multi-threshold analysis. RabbitTClust2 utilizes incremental clustering to integrate 1,000 new sequences into a pre-clustered dataset of 194,000 genomes within 1 minute, a process that results in a 22.7× speedup over RabbitTClust. In addition, we used RabbitTClust2 to generate a series of clustering results with Mash distance thresholds ranging from 0.01 to 0.2 (a total of 20 values) within 7 minutes on the RefSeq bacterial dataset. The results showed that when the clustering threshold approached 0.1, the cluster compositions changed significantly, suggesting that 0.1 may represent a critical threshold for genus-level classification in bacteria. RabbitTClust2 is available at https://github.com/RabbitBio/RabbitTClust.**

***Index Terms*—clustering, sketch, distributed computing platforms, HPC, species delineation.**

I. Introduction

Clustering genome sequences is a fundamental step in bioinformatics, essential for species boundary delineation [1] and genome deduplication [2]. The rapid advancement of high-throughput sequencing technologies [3] has driven an exponential increase in genomic data. By January 2025, NCBI GenBank has archived more than 2.5 million genome assemblies, while the RefSeq collection encompasses approximately 395,000 genomes representing over 28,000 species [4]. Performing clustering at such a scale has become a critical challenge, which can be attributed to two key factors. First, genome sequences are often extremely long, making traditional similarity computation methods inefficient for pairwise distance calculations. Second, the large number of genomes results in a vast number of pairwise comparisons, which not only requires excessive memory resources but also leads to prohibitive computation time. This issue is exacerbated by analyses that require incremental clustering (e.g., genome database maintenance) or multi-threshold clustering (e.g., taxonomic boundary detection). Thus, time-consuming alignment-based algorithms [5] often cannot compute pairwise similarities within a reasonable time. As a consequence many widely used clustering tools, such as CD-HIT [6], Linclust [7] and UCLUST, are not well suited for clustering large-scale long genomic sequences [8].

To overcome this challenge, several approaches have explored alignment-free algorithms [9] and sketching strategies [10] for estimating sequence similarities. Gclust measures sequence similarities using a sparse suffix array and maximal exact matches [11]. MeShClust3 employs a machine learning-based method to predict sequence similarity (identity scores) by analyzing their k-mer histograms [12]. iDeLUCS introduces a deep learning-based method that clusters DNA sequences by detecting genomic signatures [13]. Despite the performance improvements achieved by these tools, they still cannot efficiently cluster large-scale genomic datasets, such as GenBank (exceeding 2.5 million genomes in total). To address this issue, RabbitTClust introduced a MinHash-based approach for hierarchical clustering, enabling clustering millions of genomes [14].

However, it still faces scaling limitations when dealing with even larger datasets and with high-frequency data pro-

† These authors contributed equally to this work.
* Corresponding authors

cessing scenarios. Although RabbitTClust employs a high-performance MinHash algorithm [15], it remains suboptimal in terms of both runtime and memory efficiency, leaving substantial room for improvement. Additionally, RabbitTClust only supports multi-threading on a single node, limiting its scalability on distributed computing clusters. Furthermore, it provides only basic clustering functions and lacks support for efficient incremental and multi-threshold clustering, reducing its flexibility. For example, when performing incremental clustering with RabbitTClust, the entire dataset must be re-processed, resulting in significant computational overhead.

In this paper, we address these limitations by introducing RabbitTClust2 – a more efficient, scalable, and versatile version of RabbitTClust. We integrate a more efficient k-mer substring space decomposition (KSSD) algorithm [16], and employ an inverted index structure to accelerate the similarity computation between sequences [17]. Additionally, we incorporate an effective pre-pruning strategy to further speed up the clustering process. Moreover, we have implemented an MPI-based parallel clustering algorithm to significantly enhance processing capability in multi-node environments. RabbitTClust2 now supports fast incremental and multi-threshold clustering by saving intermediate results. This eliminates the need for users to re-run the entire clustering process, significantly improving efficiency for updates. RabbitTClust2 can efficiently cluster 2.6 million genomes in approximately one hour on a distributed cluster using 16 nodes, achieving a speedup of 43.7× over the single-node RabbitTClust and at least two orders of magnitude faster than other existing tools, such as Gclust or Mothur.

II. Methods

A. Overview of RabbitTClust2

RabbitTClust2 is an efficient and scalable clustering tool designed for large-scale genomic datasets. As illustrated in Figure 1a, it adopts a modular two-phase workflow: sketch construction and minimum spanning tree (MST) generation. In the first phase, RabbitTClust2 constructs sketches using a KSSD-based method. In the MST generation phase, it computes pairwise similarities using an inverted index, builds the MST with pruning strategies, and generates clustering results based on its structure. These optimizations significantly accelerate the clustering process. In addition, RabbitTClust2 supports execution on distributed clusters, allowing the tool to handle millions of genomes efficiently. Furthermore, RabbitTClust2 supports flexible clustering operations including incremental clustering and multi-threshold clustering, as shown in Figure 1b and Figure 1c. Incremental clustering enables the integration of new genomes into existing clustering results by reusing pre-computed sketches and MSTs. Multi-threshold clustering allows a single MST to be reused to efficiently generate clustering results under various distance cutoffs. These features significantly reduce redundant computations and enhance usability for large-scale exploratory analysis.

B. Sketch construction optimization

To further improve the efficiency of sketch construction for large-scale genomic datasets, we integrate the KSSD algorithm into RabbitTClust2 as a replacement for the classical MinHash approach. This algorithm randomly shuffles the k-mer subsequence space as a shuffled dictionary and divides it into $2^{4\times drlevel}$ parts, where *drlevel* is referred to as the dimensionality reduction parameter, with a default value of 3. Only one part is selected as the index to filter valid k-mer hash values, which is subsequently used to construct the sketch. Therefore, the sketch generated by this algorithm is variable in length, with its size adapting to the input genome. This makes it particularly suitable for distance estimation between genomes with large size discrepancies. During sketch construction, we encode k-mers using a direct character-to-bit mapping scheme, rather than applying traditional hash functions, since it is computationally faster than general-purpose hashing. This is because it involves a simple lookup operation per base, while hashing algorithms require multiple, complex mixing operations for each input byte. In addition, the process of retrieving all k-mers from the shuffled dictionary to identify valid hashes introduces a significant memory access bottleneck. To address this issue, we construct a *map dictionary* using a Robin Hood hash table, which exclusively stores the valid hashes [18]. Due to its substantially smaller size compared to the shuffled dictionary, the *map dictionary* exhibits reduced retrieval latency and superior cache locality. We redesign and integrate the KSSD algorithm to eliminate the dependency on external shuffle dictionary files. This further simplifies the usage and improves overall performance. The *fast* option is provided to enable the KSSD-based sketch generation module.

C. MST generation optimization

The MST generation phase consists of two steps: first, computing all pairwise sequence similarities, and then generating the MST based on these similarities using Kruskal's algorithm. In our multithreaded implementation, each thread processes a subset of the data to generate a corresponding sub-MST, and all sub-MSTs are finally merged into a complete global MST.

We introduce two key optimizations to enhance this phase. During the similarity computation stage, the traditional pairwise intersection method used in RabbitTClust is replaced with an inverted index, significantly accelerating the calculation. The all-to-all distance computation exhibits a time complexity of $\mathcal{O}(N^2 \cdot S)$, where N is the number of genomes and S represents the sketch size. The inverted index maps each unique hash value to the set of sketches in which it appears, as illustrated in Figure 1a. The construction of this inverted index has a time complexity of $\mathcal{O}(N \cdot s)$, where s denotes the average number of hash values per KSSD sketch. By iterating over this inverted index, we efficiently tally all co-occurrences and compute intersection sizes for all sketch pairs in a single pass. Let v be the average number of genomes that can be retrieved by each valid hash value; the time complexity of computing similarity using the inverted index is then $\mathcal{O}(v \cdot N \cdot s)$. Under

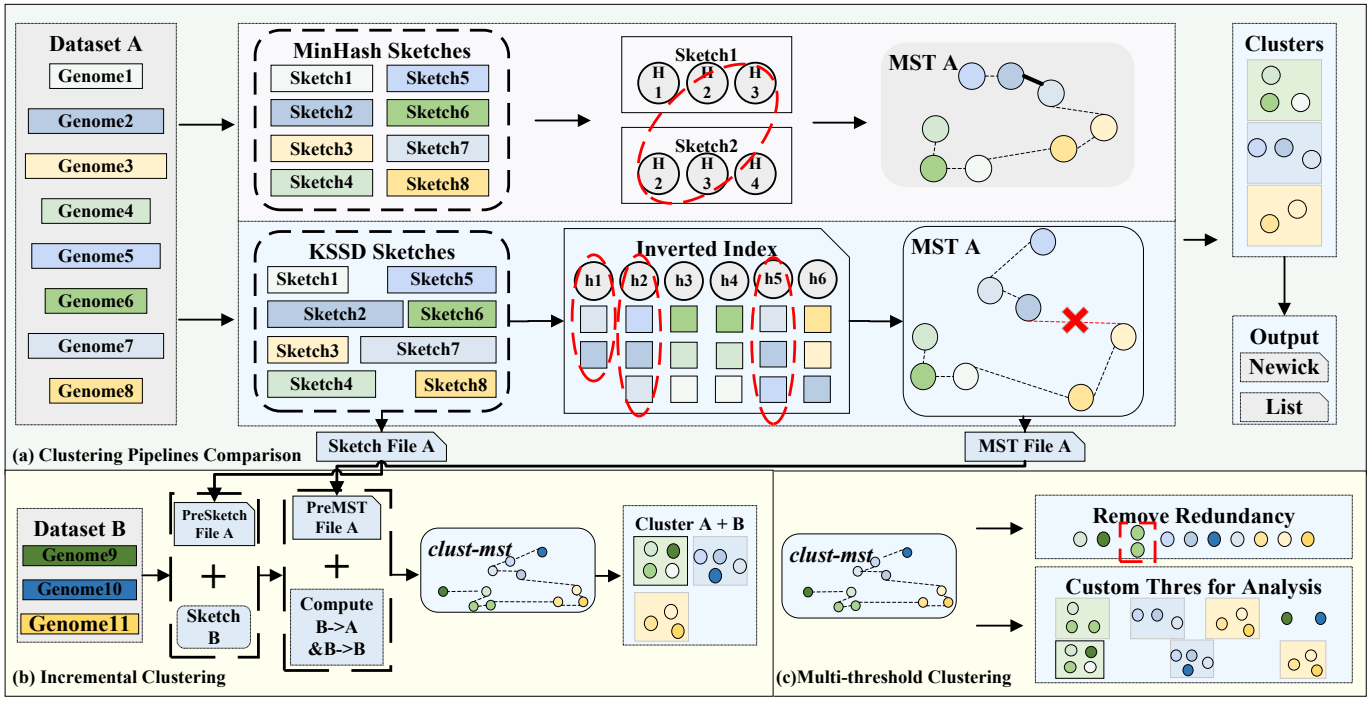


Fig. 1: Overview of our clustering workflows. (a) Comparison of clustering pipelines between RabbitTClust2 and RabbitTClust. (b) Incremental clustering using pre-computed sketches and MST files. (c) Multi-threshold clustering using pre-computed MST files.

the assumption that S and s are approximately the same, we note that $(v+1)\cdot N \ll 2\cdot N^2$. This demonstrates that the inverted-index-based method has significantly lower computational complexity compared to the pairwise approach used in RabbitTClust.

Second, we propose a heuristic pruning strategy that exploits the *sketch length ratio*. Unlike the fixed-size MinHash sketches, KSSD generates sketches with variable lengths. By leveraging the mathematical relationship between the Mash distance D and the Jaccard similarity J,

$$D = -\frac{1}{k}\ln\left(\frac{2J}{1+J}\right),$$

we can derive a theoretical upper bound for J under a given distance threshold D_0:

$$J \le \frac{e^{-kD_0}}{2-e^{-kD_0}}.$$

Since the Jaccard similarity is also bounded by the ratio of sketch lengths, we derive the pruning condition:

$$\frac{\max(|A|,|B|)}{\min(|A|,|B|)} > \frac{2-e^{-kD_0}}{e^{-kD_0}},$$

in which case the distance computation between the two sketches is skipped. This heuristic significantly reduces computational overhead without compromising clustering accuracy.

D. Distributed implementation

RabbitTClust2's single-server efficiency is fundamentally constrained by hardware limitations such as I/O bandwidth. To address the challenge of the continuing data growth, we have extended RabbitTClust by integrating Message Passing Interface (MPI) for deployment on distributed computing platforms. For multi-node parallelism, the computation tasks are partitioned according to the available computational resources to achieve load balancing. In addition, inter-node communication overhead is also taken into account.

During the sketch construction process, each data partition can be processed independently in an embarrassingly parallel manner. Therefore, the genome dataset is evenly partitioned across compute nodes (e.g., datasets A, B, and C) to enable parallel computation, as shown in Figure 2. Node A is designated as the master node. In addition to performing sketch construction, it is also responsible for receiving sketch data from other nodes and combining all the sketches. Then the complete sketch set is saved to a file on the parallel file system. Communication among the distributed nodes is facilitated by MPI. Throughout this process, communication occurs only when the master node gathers the sketch set. Furthermore, since the size of the sketch data is significantly smaller than the original genomic data, the time overhead for combining them is negligible compared to the sketch construction time. In the subsequent MST generation process, to reduce communication cost, each node independently constructs an inverted index from the sketch set file. We employ cyclic distribution strategy

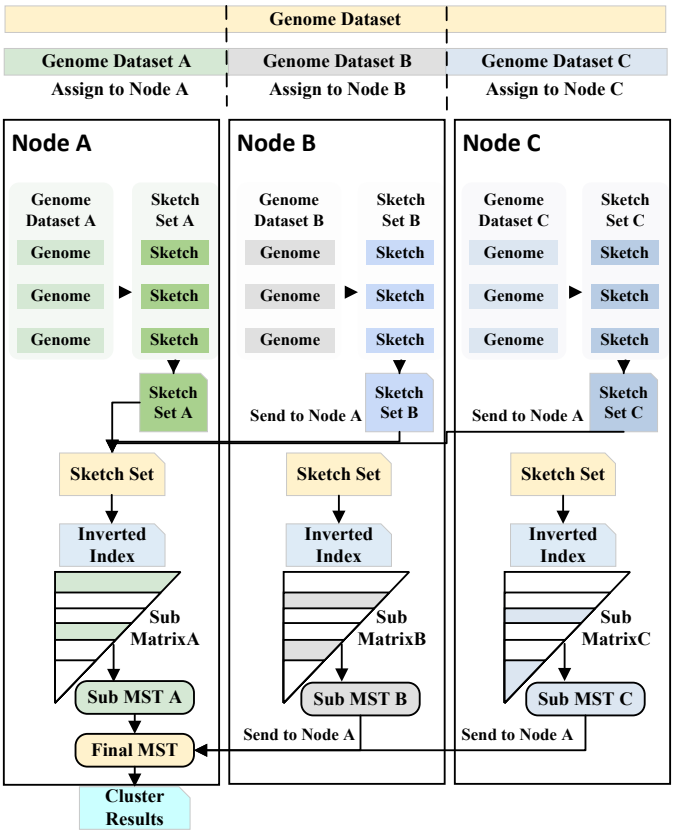


Fig. 2: Distributed implementation of RabbitTClust2 for large-scale genome clustering.

for the lower-triangular distance matrix. This strategy provides better load balancing, especially when the computational cost of distance calculation varies across different rows. Each process computes its assigned rows, builds a corresponding sub-MST, and sends the result back to the master node. The master then merges all sub-MSTs to form the final MST and derives the final clustering result. This distributed MST phase requires only one communication step: a final collection of the sub-MST. The overall communication overhead is also negligible as the amount of data transmitted during this phase is small.

E. New features in RabbitTClust2

We have integrated several new features into RabbitTClust2, including incremental clustering, multi-threshold clustering, and Newick tree generation. RabbitTClust2 features a modular design that separates sketch construction from downstream MST generation, allowing for the storage and reuse of intermediate output in subsequent analyses. By default, RabbitTClust2 writes the genome sketches to disk after the sketching phase and saves the MST once its construction is complete. Users can disable this behavior by specifying the *no-save* option to prevent the saving of intermediate files. To maximize computational efficiency, RabbitTClust2 offers several features that reuse these intermediate results (Figure 3). For example, the

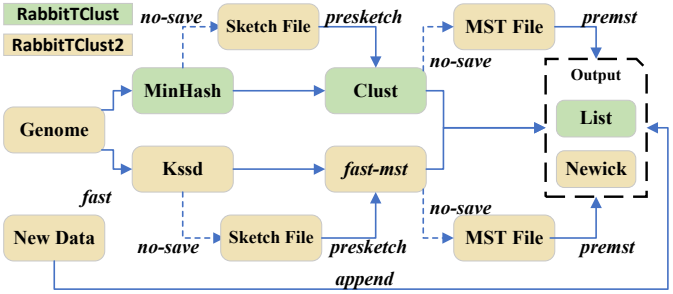


Fig. 3: Feature comparison between RabbitTClust and RabbitTClust2.

presketch feature allows users to load pre-computed sketches, skipping the often time-consuming sketch construction step. By reusing intermediate results, RabbitTClust2 can efficiently perform incremental clustering and multi-threshold clustering.

Multi-threshold clustering. We recognize that multi-threshold clustering requires rerunning the entire process to obtain optimal clustering results. While reusing sketches is beneficial, generating the MST for each threshold can still be computationally expensive. To address this bottleneck, RabbitTClust2 introduces the *premst* feature. This feature allows users to load a pre-computed MST file that represents the complete, threshold-independent relationship of all genomes. The new clustering results for any given threshold can then be generated in seconds by cutting the edges of this existing MST, as shown in Figure 1c. As a result, the analyses that would otherwise may require months of computation can now be completed efficiently.

Incremental clustering. In addition, RabbitTClust2 supports efficient incremental clustering, allowing new genomes to be added to an existing clustered dataset without full recomputation. As illustrated in Figure 1b, when a new dataset (B) is added to a pre-processed dataset (A), the tool first generates sketches for B. It then calculates only the necessary pairwise distances: those within B and those between A and B. Finally, it merges the newly computed MSTs with the pre-existing MST from dataset A to produce the final combined tree. This workflow is managed through the *append* feature, which works in conjunction with *presketch* and *premst* to reuse the intermediate files from the original dataset.

Furthermore, RabbitTClust2 introduces support for the Newick tree format, addressing a limitation of the flat cluster lists produced by RabbitTClust. The Newick output preserves the rich hierarchical relationships embedded in the MST, making the clustering results easy to visualize and compatible with other common bioinformatics tools [19]. This feature is activated using the *newick-tree* option.

III. Results

We compare RabbitTClust2 with the latest versions of Gclust (v1.0), and Mash (v2.3) & Mothur (v1.48.3) as well as to RabbitTClust to evaluate its performance. The datasets used in this study are summarized in Table I. Our experiments

are conducted on the latest NCBI GenBank bacteria (Gen-Bac, Release 266) and RefSeq bacteria (Ref-Bac, Release 230) datasets. Gen-Bac includes 2,709,910 FASTA-formatted genome sequences (9.7 TB total), while Ref-Bac consists of 195,631 genomes (820 GB total). Furthermore, to simulate incremental clustering, we created a subset of the Ref-Bac dataset, referred to as Sub-Bac (1,000 genomes, 3.4 GB in FASTA). In addition, to provide a comprehensive performance comparison reflecting RabbitTClust's support for multiple vectorization instruction sets, we conducted tests on both an Intel server (AVX512 support) and an AMD server (AVX2 support). The performance of the distributed implementation is also evaluated on a distributed platform consisting of 16 compute nodes. The detailed configurations are shown in Table II.

The accuracy of our clustering method is assessed using a ground truth constructed from the Ref-Bac dataset. Specifically, species-level classifications are extracted using the *species_taxid* from the *assembly_summary_refseq* file, while genus-level classifications can be found by mapping the *species_taxid* to the *nodes.dmp* file. Both of these files are available for download from NCBI. To measure the similarity between the predicted clusters and the ground truth, we use Normalized Mutual Information (NMI). As a metric based on information theory, NMI is widely used in clustering evaluation because it effectively quantifies the degree of shared information between two sets of labels. Additionally, we introduce precision and recall measures to evaluate the clustering performance. Unlike standard classification tasks, we consider whether the tool clusters similar elements together and whether it avoids separating similar elements. As a result, the definitions of True Positive (TP), False Positive (FP), and False Negative (FN) in clustering tasks differ from those in classification tasks. The specific definitions are as follows:

- **TP**: Two elements are assigned to the same cluster and also belong to the same class in the ground truth labels.
- **FP**: Two elements are assigned to the same cluster but belong to different classes in the ground truth labels.
- **FN**: Two elements belong to the same class in the ground truth labels but are assigned to different clusters.
- **TN**: Two elements belong to different classes in the ground truth labels and are also assigned to different clusters.

These four values are computed using a confusion matrix, from which we further derive precision, recall, and F1-score.

TABLE I: Description of datasets.

Dataset	Description	File Number	Data Size
Gen-Bac	the Bacteria dataset in GenBank Release 266	2,709,910	9.7 TB
Ref-Bac	the Bacteria dataset in RefSeq Release 230	195,631	820 GB
Sub-Bac	the subset of Ref-Bac	1,000	3.4 GB

A. Performance for Large-Scale Genome Clustering

Table III presents the performance comparison between RabbitTClust2 and other tools using the Ref-Bac dataset across both Intel and AMD platforms. RabbitTClust2 achieves the best overall performance, maintaining high clustering accuracy (e.g. NMI = 0.956, F1 = 0.922) while significantly reducing runtime and memory usage. On the Intel platform, RabbitTClust2 completes the full clustering process in only 248.9 seconds, which is 4.5× faster than RabbitTClust (1108.0 s) and 276× faster than the Mothur & Mash pipeline (68719.5 s), while also using less memory (14.8 GB) compared to RabbitTClust (18.8 GB). Furthermore, RabbitTClust2 outperforms RabbitTClust consistently on the AMD platform as well, reducing total runtime from 869.1 seconds to 367.8 seconds and memory consumption from 25.8 GB to 22.7 GB. In contrast, MeshClust and Gclust could not complete the task within memory constraints (OOM on both servers). To provide a more comprehensive comparison, we conducted experiments on the Sub-Bac dataset, with the results shown in Table IV. Due to the time required for inverted index construction, RabbitTClust2 does not demonstrate a significant advantage over RabbitTClust on smaller datasets. However, RabbitTClust2 achieves the highest clustering accuracy among all compared tools. These results demonstrate that RabbitTClust2 offers substantial improvements in terms of efficiency without compromising clustering quality, making it highly suitable for large-scale genome datasets.

Table V presents the clustering performance on the 9.7 TB Gen-Bac dataset. Compared to RabbitTClust, RabbitTClust2 achieves a 2.0× speedup in total runtime (reducing from 51.62 to 26.18 hours) while cutting peak memory usage nearly in half (from 223.3 GB to 113.7 GB). These improvements are the result of our comprehensive optimizations of both the sketching and MST construction stages. In the sketching phase, RabbitTClust uses the MinHash algorithm to generate a fixed-size sketch of 8 KB (1,000 hash values) per genome, regardless of genome size. In contrast, RabbitTClust2 employs the KSSD algorithm, which produces sketches proportional to the genome length, for instance, a 1 MB genome yields a 1 KB sketch. Consequently, KSSD sketches remain smaller than

TABLE II: Software and Hardware Configuration of the Test Platforms

Environment	AMD Server	Intel Server	Distributed Platform
Nodes	1	1	16
CPU	AMD EPYC 7T83	Intel Xeon Platinum	AMD EPYC 7452
Sockets	2	2	2
Cores	2x64	2x32	2x32
RAM	256 GB	256 GB	256 GB
Compiler	GCC 8.5.0	GCC 8.5.0	GCC 4.8.5
MPI Library	N/A	N/A	Intel MPI 2022.1
Interconnect	N/A	N/A	56 Gb/s FDR InfiniBand

TABLE III: Performance Comparison for RabbitTClust2 in Ref-Bac Dataset.

Platform	Tool	NMI	Recall	Precision	F1	Memory	Sketch Phase	Clustering Phase	Total	Speedup
Intel	Gclust	-	-	-	-	OOM	-	-	-	-
	Mothur & Mash	0.953	0.835	0.894	0.863	19.6 GB	376.1 s	68343.4 s	68719.5 s	276.1
	RabbitTClust	0.956	0.997	0.857	0.922	18.8 GB	294.8 s	813.2 s	1108.0 s	4.5
	RabbitTClust2	0.956	0.997	0.856	0.922	14.8 GB	118.5 s	130.4 s	248.9 s	1.0
AMD	Gclust	-	-	-	-	OOM	-	-	-	-
	Mothur & Mash	0.953	0.835	0.894	0.863	27.7 GB	394.3 s	64290.4 s	64684.7 s	175.9
	RabbitTClust	0.956	0.997	0.857	0.922	25.8 GB	290.6 s	578.4 s	869.1 s	2.4
	RabbitTClust2	0.956	0.997	0.856	0.922	22.7 GB	287.7 s	80.1 s	367.8 s	1.0

TABLE IV: Performance Comparison for RabbitTClust2 in Sub-Bac Dataset.

Platform	Tool	NMI	Recall	Precision	F1	Memory	Sketch Phase	Clustering Phase	Total	Speedup
Intel	Gclust	0.829	0.250	0.977	0.398	40.3 GB	-	-	2034.2 s	274.9
	Mothur&Mash	0.986	0.948	0.801	0.868	4.9 GB	2.9 s	1.2 s	4.1 s	0.6
	RabbitTClust	0.986	0.948	0.801	0.868	1.3 GB	1.3 s	0.1 s	1.4 s	0.2
	RabbitTClust2	0.987	0.948	0.802	0.869	7.8 GB	1 s	6.4 s	7.4 s	1.0
AMD	Gclust	0.829	0.250	0.977	0.398	43.8 GB	-	-	1870.0 s	275.0
	Mothur&Mash	0.986	0.948	0.801	0.868	6.1 GB	2.1 s	1.1 s	3.2 s	0.5
	RabbitTClust	0.986	0.948	0.801	0.868	2.3 GB	2.1 s	0.2 s	2.3 s	0.3
	RabbitTClust2	0.987	0.948	0.802	0.869	8.3 GB	1.2 s	5.6 s	6.8 s	1.0

TABLE V: Performance Comparison for Large-Scale Genome Clustering in Gen-Bac Dataset.

Tool	Memory	Sketch Phase	Clustering Phase	Total	Speedup
RabbitTClust	223.3 GB	0.98 h	50.64 h	51.62 h	2.0
RabbitTClust2	113.7 GB	0.37 h	25.81 h	26.18 h	1.0

MinHash sketches for genomes under 8 MB, which comprise the majority of the Gen-Bac dataset. After sketching the entire dataset, the total sketch size generated by RabbitTClust2 is only 43% of that produced by RabbitTClust. Moreover, during the MST generation stage, RabbitTClust performs all pairwise distance computations in memory, requiring full sketch loading. RabbitTClust2, by contrast, uses a compact inverted index to avoid loading all sketches simultaneously. This significantly reduces memory usage and enables RabbitTClust2 to scale efficiently on large datasets.

B. Performance on Distributed Platforms

The scalability and performance of RabbitTClust2 on a distributed platform are illustrated in Figure 4 and Figure 5. Figure 4 presents the scalability results on the Ref-Bac dataset, where the number of nodes scales from 1 to 16. RabbitTClust2 achieves nearly linear speedup up to 8 nodes, and continues to gain performance at 16 nodes, demonstrating its high parallel efficiency on distributed systems. This efficiency benefits from our distribution strategy, which uniformly assigns the pairwise similarity computations across all nodes using the cyclic distribution strategy, achieving effective load balancing and maximizing node utilization. However, as the number of nodes increases, the portion of time spent on inverted index

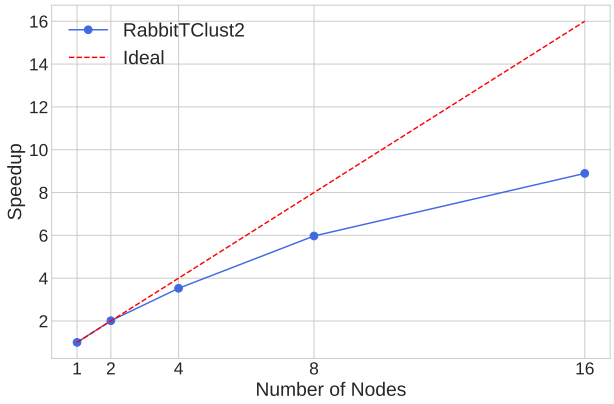


Fig. 4: Scalability of RabbitTClust2 on the Distributed Platform.

construction becomes more prominent, eventually forming a performance bottleneck that limits further scalability.

In addition, as shown in Figure 5, RabbitTClust2 significantly outperforms the original single-node RabbitTClust when processing the large-scale Gen-Bac dataset, reducing the total runtime from 51.62 hours to only 1.18 hours using 16 nodes, corresponding to a speedup of 43.7×. Both the sketching and clustering phases benefit from the distributed design, indicating the effectiveness of the parallelization strategy in practical large-scale genomic clustering scenarios.

C. Performance of incremental clustering

To simulate an incremental clustering scenario, we partition the Ref-Bac dataset into two subsets: the Sub-Bac as an incremental set, and the remaining about 194,000 genomes as a base set. The performance comparison is summarized

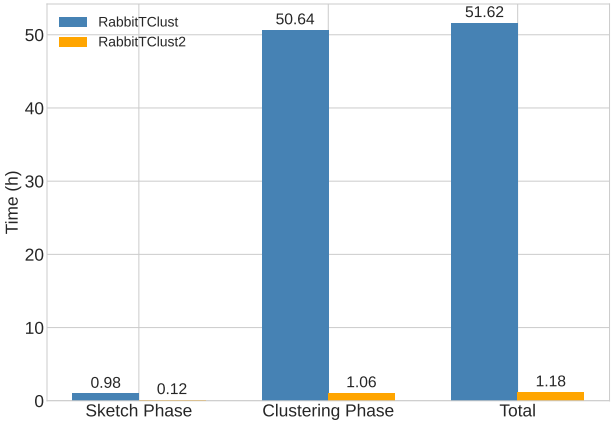
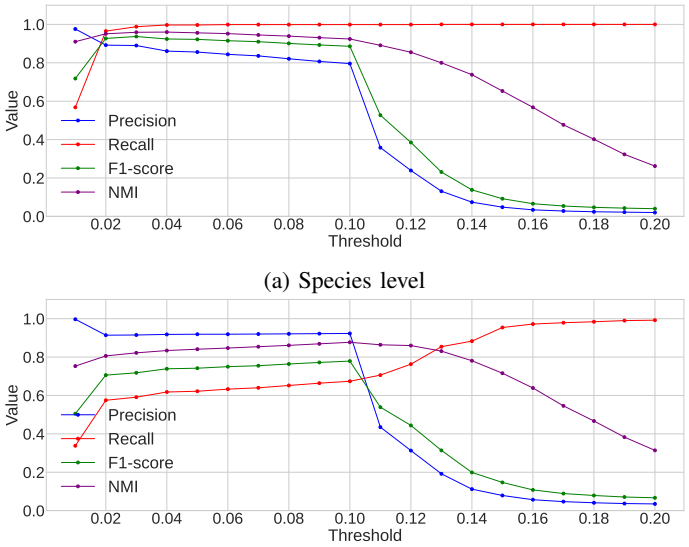


Fig. 5: Performance Comparison between RabbitTClust and RabbitTClust2 on the Gen-Bac Dataset.



(a) Species level

(b) Genus level

Fig. 6: Evaluation of RabbitTClust2 clustering performance at species and genus levels

in Table VI. In the sketching phase, RabbitTClust requires reprocessing the entire dataset, resulting in a sketching time of 294.8 seconds. In contrast, RabbitTClust2 only sketches the newly added genomes, completing the task in just 0.9 seconds, achieving a 327.6× speedup. During the clustering phase, RabbitTClust performs a full all-to-all pairwise distance computation, taking 813.2 seconds. In comparison, RabbitTClust2 leverages previously stored results and only computes the necessary distances involving the new genomes, completing clustering in 47.9 seconds. Overall, RabbitTClust2 reduces the total runtime from 1108.0 to 48.8 seconds, achieving a 22.7× speedup. These results highlight the effectiveness of RabbitTClust2 in handling incremental clustering tasks.

D. Performance of multi-threshold clustering

Table VII presents a performance comparison between RabbitTClust and RabbitTClust2 for multi-threshold clustering on the Ref-Bac dataset using 20 different distance thresholds. RabbitTClust2 completes the entire process in 384.9 seconds, whereas RabbitTClust takes 9830.4 seconds, resulting in a 25.5× overall speedup. This substantial improvement is primarily due to RabbitTClust2's pre-MST strategy, which performs a single full clustering at the base threshold (e.g., 0.2) and reuses the computed minimum spanning tree (MST) structure to efficiently derive clustering results for the remaining 19 thresholds. The additional computation required for generating these results is minimal, approximately 17 seconds in total. In contrast, RabbitTClust must independently perform 20 full clustering executions, one for each threshold, as it does not support result reuse across thresholds. This leads to significantly higher computational overhead and makes it unsuitable for the multi-threshold clustering scenario. These results clearly demonstrate that RabbitTClust2 is well-suited for multi-threshold clustering tasks, enabling fast, scalable, and flexible analysis across varying similarity criteria.

TABLE VI: Performance Comparison for Incremental Clustering on the Sub-Bac Dataset.

Tool	Sketch Phase	Clustering Phase	Total	Speedup
RabbitTClust	294.8 s	813.2 s	1108.0 s	22.7
RabbitTClust2	0.9 s	47.9 s	48.8 s	1.0

TABLE VII: Performance Comparison for Multi-threshold Clustering in Ref-Bac Dataset.

Tool	Dataset	#Thresholds*	Total	Speedup
RabbitTClust	Ref-Bac	[0.01, 0.2] (n=20)	9830.4 s	25.5
RabbitTClust2	Ref-Bac	[0.01, 0.2] (n=20)	384.9 s	1.0

* #Thresholds indicate the range and number of clustering thresholds used in the experiment.

E. Species and Genus-Level Boundary Detection

To investigate whether clear genome similarity boundaries exist at the species and genus levels, we conducted a large-scale clustering experiment using RabbitTClust2 on the 820 GB Ref-Bac dataset, which contains 195,631 bacterial genomes. Clustering is performed across a range of Mash distance thresholds (0.01 to 0.2), and the results are evaluated using taxonomic labels at both species and genus levels (Figure 6).

At both levels, we observe a consistent pattern: increasing the threshold from 0.01 to 0.02 leads to a sharp drop in precision, accompanied by significant improvements in recall, F1-score, and NMI. This is expected, as a threshold of 0.01 is extremely conservative, many genomes from the same taxon are not clustered together, resulting in low recall, while the few clusters that do form are highly pure, producing near-perfect precision. As the threshold relaxes, more correct pairs are clustered, increasing recall, but also introducing more false positives, which lowers precision.

At the species level, recall stabilizes above 0.99 beyond a threshold of 0.03, yet precision remains relatively low across the entire range. This suggests that although RabbitTClust2 successfully groups genomes from the same species, it struggles to prevent merging between highly similar but taxonomically distinct species. This challenge arises in part from the fact that high sequence similarity does not always imply taxonomic identity [1], and also from the inherent approximation of sketch-based clustering. At the genus level, we observe a clearer separation. Precision remains consistently high (over 0.92) between thresholds 0.02 and 0.1, and recall steadily improves, reaching 0.67 at its peak. However, even at a threshold of 0.1, not all genuslevel groups are fully merged, indicating that the boundary remains conservative.

At threshold = 0.11, we observe a sharp inflection: recall increases abruptly as previously fragmented clusters merge, but precision simultaneously drops at both species and genus levels. This indicates that the threshold of 0.1–0.11 represents a critical tipping point, where inter-genus merging becomes frequent due to transitive connections in the MST graph. This behavior suggests that bacterial genus-level boundaries are associated with a Mash distance around 0.1, beyond which taxonomic structure rapidly collapses into over-merged clusters. The MST structure plays a key role: once a single edge links two otherwise distinct taxa, the entire clusters may fuse, leading to a cascade effect that blurs taxonomic distinctions.

These findings support the existence of a genome similarity boundary at the genus level around Mash distance 0.1. For species-level clustering, we recommend using 0.03 as a default threshold to balance precision and recall. For exploratory analysis aiming to maximize recall, 0.1 can be used, though it may result in over-merging of closely related species.

IV. Conclusions

We have presented RabbitTClust2, a highly efficient and versatile tool for large-scale genome clustering. By integrating a highly efficient sketching algorithm with an optimized MST generation process that leverages pruning strategies and an inverted index for distance computation, we significantly improve clustering performance. Moreover, RabbitTClust2 has been extended to distributed computing platforms, enabling the efficient clustering of 2.6 million genomes in about one hour. In addition, it offers expanded output format options and supports efficient incremental clustering as well as multi-threshold clustering, which allows for the detection of the genus-level boundaries within the bacteria. This makes RabbitTClust2 a well-suited tool to meet the demands of continuously growing datasets. It is publicly available at https://github.com/RabbitBio/RabbitTClust.

Acknowledgment

This work is partially supported by the Shandong University Future Scholar Program; Engineering Research Center of Digital Media Technology, Ministry of Education, China; DFG RMaP project C01.

References

[1] C. Jain, L. M. Rodriguez-R, A. M. Phillippy, K. T. Konstantinidis, and S. Aluru, "High throughput ani analysis of 90k prokaryotic genomes reveals clear species boundaries," *Nature communications*, vol. 9, no. 1, p. 5114, 2018.

[2] W. Li, L. Fu, B. Niu, S. Wu, and J. Wooley, "Ultrafast clustering algorithms for metagenomic sequence analysis," *Briefings in bioinformatics*, vol. 13, no. 6, pp. 656–668, 2012.

[3] B. Schmidt and A. Hildebrandt, "Next-generation sequencing: big data meets high performance computing," *Drug discovery today*, vol. 22, no. 4, pp. 712–717, 2017.

[4] T. Goldfarb, V. K. Kodali, S. Pujar, V. Brover, B. Robbertse, C. M. Farrell, D.-H. Oh, A. Astashyn, O. Ermolaeva, D. Haddad *et al.*, "Ncbi refseq: reference sequence standards through 25 years of curation and annotation," *Nucleic Acids Research*, vol. 53, no. D1, pp. D243–D257, 2025.

[5] T. F. Smith, M. S. Waterman *et al.*, "Identification of common molecular subsequences," *Journal of molecular biology*, vol. 147, no. 1, pp. 195–197, 1981.

[6] L. Fu, B. Niu, Z. Zhu, S. Wu, and W. Li, "Cd-hit: accelerated for clustering the next-generation sequencing data," *Bioinformatics*, vol. 28, no. 23, pp. 3150–3152, 2012.

[7] M. Steinegger and J. Söding, "Clustering huge protein sequence sets in linear time," *Nature communications*, vol. 9, no. 1, p. 2542, 2018.

[8] Z. Ju, H. Zhang, J. Zhang, W. Xi, D. Huang, S. Feng, J. Meng, and Y. Wei, "A comparative study of sequence clustering algorithms," *Big Data Mining and Analytics*, vol. 8, no. 5, pp. 1011–1022, 2025.

[9] A. Zielezinski, S. Vinga, J. Almeida, and W. M. Karlowski, "Alignment-free sequence comparison: benefits, applications, and tools," *Genome biology*, vol. 18, no. 1, p. 186, 2017.

[10] W. P. Rowe, "When the levee breaks: a practical guide to sketching algorithms for processing the flood of genomic data," *Genome biology*, vol. 20, pp. 1–12, 2019.

[11] R. Li, X. He, C. Dai, H. Zhu, X. Lang, W. Chen, X. Li, D. Zhao, Y. Zhang, X. Han *et al.*, "Gclust: a parallel clustering tool for microbial genomic data," *Genomics, proteomics & bioinformatics*, vol. 17, no. 5, pp. 496–502, 2019.

[12] H. Z. Girgis, "Meshclust v3. 0: high-quality clustering of dna sequences using the mean shift algorithm and alignment-free identity scores," *BMC genomics*, vol. 23, no. 1, p. 423, 2022.

[13] P. Millan Arias, K. A. Hill, and L. Kari, "i delucs: a deep learning interactive tool for alignment-free clustering of dna sequences," *Bioinformatics*, vol. 39, no. 9, p. btad508, 2023.

[14] X. Xu, Z. Yin, L. Yan, H. Zhang, B. Xu, Y. Wei, B. Niu, B. Schmidt, and W. Liu, "Rabbittclust: enabling fast clustering analysis of millions of bacteria genomes with minhash sketches," *Genome Biology*, vol. 24, no. 1, p. 121, 2023.

[15] B. D. Ondov, T. J. Treangen, P. Melsted, A. B. Mallonee, N. H. Bergman, S. Koren, and A. M. Phillippy, "Mash: fast genome and metagenome distance estimation using minhash," *Genome biology*, vol. 17, pp. 1–14, 2016.

[16] H. Yi, Y. Lin, C. Lin, and W. Jin, "Kssd: sequence dimensionality reduction by k-mer substring space sampling enables real-time large-scale datasets analysis," *Genome biology*, vol. 22, pp. 1–20, 2021.

[17] T. Zhang, Z. Yin, X. Xu, L. Yan, F. Zhu, X. Duan, B. Schmidt, and W. Liu, "Rabbitsketch: a high-performance sketching library for genome analysis," *Bioinformatics*, vol. 41, no. 5, p. btaf249, 2025.

[18] X. Xu, Z. Yin, L. Yan, H. Yi, H. Wang, B. Schmidt, and W. Liu, "Rabbitkssd: accelerating genome distance estimation on modern multi-core architectures," *Bioinformatics*, vol. 39, no. 11, p. btad695, 2023.

[19] M. B. Eisen, P. T. Spellman, P. O. Brown, and D. Botstein, "Cluster analysis and display of genome-wide expression patterns," *Proceedings of the National Academy of Sciences*, vol. 95, no. 25, pp. 14 863–14 868, 1998.